“Linux File Descriptor and File I/O Management System”

Operating Systems and Systems Programming

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

J. Sai Spoorthi	**2520030587**
N. Sai Pranavi	**2520030514**
T. Sudeshna	**2520030591**

Under the Esteemed Guidance of

Dr. K. Anusha
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **"Linux File Descriptor and File I/O Management System"** is a record of Bonafide work of **J. SAI SPOORTHI - 2520030587, N. SAI PRANAVI - 2520030514, T. SUDESHNA - 2520030591** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L (Deemed to be) University, as part of the Operating Systems and Systems Programming course. The results embodied in this report have not been copied from any other departments/University/Institute.

The implementation, execution results, and observations presented in this report are based on the project source code and the demonstrated terminal execution.

J. Sai Spoorthi	**2520030587**
N. Sai Pranavi	**2520030514**
T. Sudeshna	**2520030591**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled **"Linux File Descriptor and File I/O Management System"** is a bonafide work done and submitted by **J. Sai Spoorthi – 2520030587, N. Sai Pranavi – 2520030514, T. Sudeshna – 2520030591,**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Engineering (Operating Systems and Systems Programming course), K L (Deemed to be University), during the academic year **2026–27.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

We express our sincere gratitude to **Dr. K. Anusha**, Department of Computer Science and Engineering, for providing guidance and support throughout the development of this project.

We express our gratitude to the Head of the Department of Computer Science and Engineering for providing us with adequate facilities, ways and means by which we were able to complete this project.

We also thank the faculty and laboratory staff for providing the Linux/Ubuntu environment and the facilities required to implement and test the system.

Finally, we thank our classmates and family members for their encouragement and cooperation.

J. Sai Spoorthi 2520030587

N. Sai Pranavi 2520030514

T. Sudeshna 2520030591

Abstract

The Linux File Descriptor and File I/O Management System is a Linux-based systems programming project that demonstrates how an operating system manages files through file descriptors and File I/O operations. The system is implemented in C using Linux system calls and POSIX APIs. It provides a command-driven interface through which a user can create, open, write, read, and close a file, while displaying the assigned file descriptor and the number of bytes transferred.

The project is extended beyond basic file handling to demonstrate core operating-system concepts covered in the course. Process creation is implemented using fork(), and parent-child communication is demonstrated using a pipe. Process identifiers are displayed using getpid() and getppid(). Copy-on-write/process memory behavior is illustrated by modifying a variable in the child after fork() while observing that the parent's value remains unchanged. Shared data is implemented using mmap() with MAP_SHARED | MAP_ANONYMOUS, allowing parent and child processes to access the same integer. The shared increment operation also provides a practical setting for understanding race conditions because the read-modify-write sequence is not protected by a process-shared synchronization primitive.

Finally, POSIX threads are created using pthread_create(), and access to the thread-shared variable is protected using a mutex. pthread_join() is used to wait for both threads. Thus, the project connects file management, processes, inter-process communication, process memory, shared data, concurrency, and synchronization in one demonstrable Linux application.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		8–9
		1.1 Background of the Project	8
		1.2 Problem Statement	8
		1.3 Scope of the Project	8
		1.4 Objectives of the Project	8
		1.5 Importance of the Application	9
		1.6 Target Users / Audience	9
2	OS Concepts and Background Study		10–11
		2.1 Course Outcome Mapping	10
		2.2 File Descriptors and File I/O	10
		2.3 Process Creation and Process IDs	10
		2.4 Pipe Communication	10
		2.5 Copy-on-Write and Process Memory	11
		2.6 Shared Data and Race Condition	11
		2.7 Threads, Concurrency and Synchronization	11
3	System Requirements		12
		3.1 Hardware Requirements	12
		3.2 Software Requirements	12
		3.3 Operating System Environment – Ubuntu / WSL	12
		3.4 Development Tools and Compilers	12
4	Technology and Tools Used		13
		4.1 C Programming	13
		4.2 GCC Compiler	13
		4.3 Linux System Calls and APIs	13
		4.4 POSIX Threads	13
		4.5 Nano and Terminal	13
5	System Architecture and Design		14–16
		5.1 Overall System Architecture	14
		5.2 System Workflow	14

		5.3 Flowchart / Data Flow	**14**
		5.4 Module Description	**15**
		5.5 Input and Output Design	**16**
6	**Implementation**		**17–19**
		6.1 File Management Implementation (CO-1)	**17**
		6.2 Process and Pipe Implementation (CO-2 and CO-3)	**17**
		6.3 Process Memory Implementation (CO-4)	**17**
		6.4 Shared Data Implementation (CO-5)	**17**
		6.5 Thread Implementation (CO-6)	**18**
		6.6 Algorithms / Pseudocode	**18**
		6.7 CO-wise Implementation Mapping	**19**
7	**Execution and Demonstration**		**20–22**
		7.1 Compilation in Linux	**20**
		7.2 Sample Execution	**20**
		7.3 Demonstration of OS Concepts	**22**
		7.4 Output Analysis	**22**
8	**Results, Conclusion and Future Scope**		**23–24**
		8.1 Results	**23**
		8.2 Learning Outcomes	**23**
		8.3 Conclusion	**23**
		8.4 Future Scope	**23**
9	**References**		**25**
	Appendix A	**Source Code**	**26**

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Operating systems provide the interface between application programs and computer hardware. In Linux, applications interact with files and other resources through system calls. A file descriptor is a small integer associated with an open file within a process and is used by system calls such as `read()`, `write()`, and `close()`.

The project started from the Linux File Descriptor and File I/O Management System proposed for the course. The submitted project problem statement describes the objective of showing how Linux assigns file descriptors and how read, write, and close operations are performed through system calls. The implemented program extends this foundation with process, IPC, memory, shared-data, and thread demonstrations.

1.2 Problem Statement

Beginners often learn operating-system concepts separately: file descriptors in one topic, processes in another, IPC in another, and synchronization later. The purpose of this project is to combine these concepts in one small Linux C program so that their runtime behavior can be observed directly from terminal output.

1.3 Scope of the Project

- File creation, opening, writing, reading, and closing using Linux system calls.
- Displaying the file descriptor assigned to an opened file.
- Creating a child process and displaying parent/child process IDs.
- Sending file information from parent to child through a pipe.
- Demonstrating separate process memory after `fork()`.
- Creating shared anonymous memory using `mmap()` and updating shared data from parent and child.
- Creating POSIX threads and protecting a shared variable using a mutex.
- Joining threads and reporting the final synchronized value.

1.4 Objectives of the Project

1. Demonstrate file descriptors and low-level Linux File I/O.
2. Understand process creation and process identifiers.

3. Demonstrate IPC using a pipe.
4. Observe process memory behavior associated with `fork()` and copy-on-write.
5. Demonstrate shared data and the possibility of race conditions.
6. Understand threads, concurrency, mutex-based synchronization, and thread joining.

1.5 Importance of the Application

The application is useful as a laboratory demonstration because the user can execute each OS concept through a simple command. The terminal output exposes identifiers, byte counts, shared values, and synchronization events rather than hiding them behind a graphical interface.

1.6 Target Users / Audience

The primary users are students learning Operating Systems, Linux system programming, C programming, process management, IPC, shared memory, and POSIX threads.

CHAPTER -2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Course Outcome Mapping

CO	Concept	Project Command / Module	Main APIs
CO-1	File I/O + File Descriptor	create, open, write, read, close	open(), read(), write(), close(), lseek()
CO-2	Process Creation + Process IDs	process	fork(), getpid(), getppid(), wait()
CO-3	Pipe Communication	process	pipe(), read(), write()
CO-4	Copy-on-Write / Process Memory	memory	fork(), wait()
CO-5	Shared Data + Race Condition	shared	mmap(), fork()
CO-6	Threads + Concurrency + Synchronization	execute	pthread_create(), pthread_mutex_lock(), pthread_join()

2.2 File Descriptors and File I/O

The `create` command uses `open()` with `O_WRONLY | O_CREAT | O_TRUNC` to create or recreate a file. The `open` command opens the selected file with `O_RDWR` and stores the returned descriptor in `fd`. The `write` command transfers user input to the file through `write()`. Before reading, `lseek()` moves the file offset to the beginning. The `read` command then obtains the contents using `read()`. The `close` command releases the descriptor using `close()`.

2.3 Process Creation and Process IDs

The `process` command creates a child with `fork()`. The parent receives the child's process ID as the return value of `fork()`, while the child receives zero. `getpid()` reports the current process ID and `getppid()` reports the parent process ID. `wait()` allows the parent to wait for the child and inspect its exit status.

2.4 Pipe Communication

A pipe provides a unidirectional communication channel represented by two file descriptors: a read end and a write end. In this project, the parent closes the read end, formats the file name and descriptor into a message, and writes the message to the pipe. The child closes the write end, reads the message, and prints it. This demonstrates inter-process communication using a kernel-managed pipe.

2.5 Copy-on-Write and Process Memory

The `memory` command initializes `value` to 100 and then calls `fork()`. The child changes its value to 200, while the parent later prints its own value as 100. This demonstrates the process-memory behavior expected after `fork()`: parent and child do not normally modify the same private variable merely because it existed before `fork()`. The implementation therefore provides a practical demonstration of the copy-on-write concept.

2.6 Shared Data and Race Condition

The `shared` command creates an integer using `mmap()` with `MAP_SHARED | MAP_ANONYMOUS`. After `fork()`, both processes access the same mapped integer. Each process performs a read-modify-write increment loop. The expression is not an atomic increment and the shared operation is not protected by a process-shared mutex or semaphore. Therefore, concurrent execution can produce a race condition and a final value lower than the theoretical expected value. In the demonstrated run, the final value happened to be 2000 for 1000 operations in each process.

Important observation: the output value 2000 is a successful observed run, not a guarantee that every run will produce 2000. The code is intentionally useful for discussing shared data and race conditions.

2.7 Threads, Concurrency and Synchronization

The `execute` command creates two POSIX threads. Both receive the address of `shared_value`. The thread function locks the mutex before incrementing and printing the shared value and unlocks it after the critical section. `pthread_join()` is then used to wait for both threads. The mutex prevents the two threads from simultaneously executing the protected critical section, so the final shared value is 2.

CHAPTER -3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Component	Requirement
Processor	Any modern x86/x64 processor
RAM	Minimum 4 GB recommended for Linux/WSL development
Storage	At least 1 GB free space for source, compiler and project files
Input	Keyboard
Output	Terminal display

3.2 Software Requirements

Software	Purpose
Ubuntu/Linux or WSL	Execution environment
C compiler (GCC)	Compilation
GNU/Linux system libraries	System calls and POSIX interfaces
Nano/Vim or any text editor	Source-code editing
Terminal	Compilation and execution

3.3 Operating System Environment – Ubuntu / WSL

The demonstrated commands were executed from a Linux shell prompt. The program uses Linux/POSIX APIs such as `fork()`, `pipe()`, `mmap()`, and pthread functions, so a Linux environment such as Ubuntu or a compatible WSL installation is appropriate.

3.4 Development Tools and Compilers

The source file is `project1.c`. It is edited using nano and compiled with GCC using the `-pthread` option because the program uses POSIX threads.

```
nano project1.c
gcc project1.c -o project1 -pthread
./project1
```

CHAPTER -4 TECHNOLOGY AND TOOLS USED

Technology / Tool	Use in Project
C Programming	Implementation of all project modules
GCC	Compiles project1.c into project1
Linux system calls	open, read, write, close, lseek, fork, pipe, mmap
POSIX process APIs	getpid, getppid, wait
POSIX Threads	pthread_create, pthread_join
Mutex	pthread_mutex_lock/unlock and thread synchronization
Nano	Editing the C source file
Linux Terminal	Build, execution, and demonstration

4.1 C Programming

C provides direct access to low-level operating-system interfaces and is therefore suitable for demonstrating file descriptors, processes, IPC, memory mapping, and threads.

4.2 GCC Compiler

GCC compiles the C source file. The `-pthread` option enables the required POSIX thread compilation/linking support.

4.3 Linux System Calls and APIs

The implementation directly uses `open()`, `read()`, `write()`, `close()`, `lseek()`, `pipe()`, `fork()`, `mmap()`, and process/thread APIs. Error paths use `perror()` in several system-call operations.

4.4 POSIX Threads

The program creates two threads with `pthread_create()`, protects a critical section with a mutex, and waits for completion using `pthread_join()`.

4.5 Nano and Terminal

Nano is used to edit `project1.c`. The terminal is used to compile and execute the project interactively.

CHAPTER -5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The application follows a menu/command-driven architecture. The `main()` function continuously reads a command and dispatches control to the corresponding module. The modules are logically organized as follows:

Module	Command	Functionality
File Management	create/open/write/read/close	File descriptor and File I/O
Process Management	process	fork, IDs, wait
IPC	process	Pipe-based parent-child communication
Process Memory	memory	fork and private process memory
Shared Data	shared	mmap shared integer and concurrent increments
Thread Management	execute	POSIX threads, mutex and join
Termination	exit	Close open file and terminate

5.2 System Workflow

1. Start program.
2. Read a command from the user.
3. Execute the selected OS module.
4. Display operation result and relevant IDs/values.
5. Return to command prompt.
6. Close the file if requested and exit when the user enters `exit`.

5.3 Flowchart / Data Flow

Figure 5.1 shows the control flow of the program. After the banner is displayed, `main()` repeatedly reads a command and dispatches it to the matching module. Every module reports its result on the terminal and returns to the prompt; only the `exit` command leaves the loop, closing any open file first.

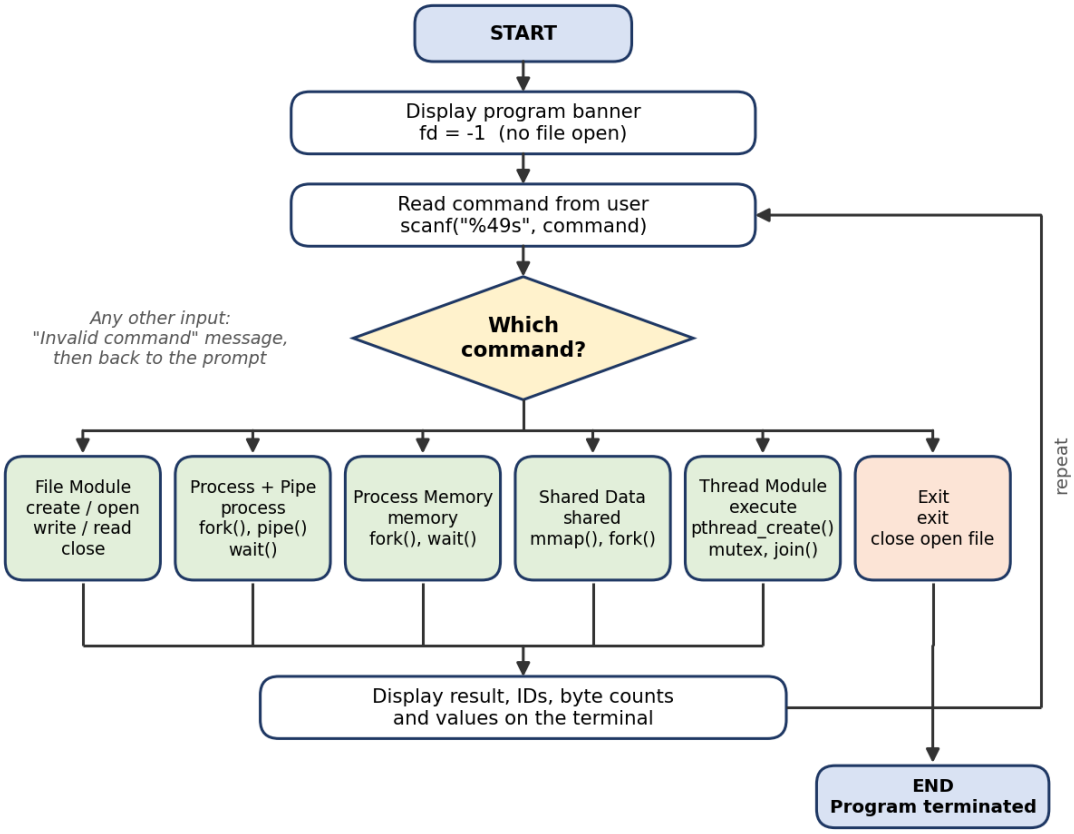


Figure 5.1: Control flow of the command-driven program

The data flow follows the same structure. The file module establishes the descriptor `fd` and the file name; when the `process` command is used, these values are formatted into a message and passed from the parent to the child through the pipe. The `shared` and `execute` commands operate on their own shared integers, one placed in mapped memory for processes and one local variable shared between threads.

5.4 Module Description

File module: Maintains `fd` and filename, performs create/open/read/write/close operations.

Process module: Creates a child process, exchanges information through a pipe, and waits for completion.

Memory module: Shows that changing a variable in the child does not change the parent's corresponding private variable.

Shared module: Maps an integer into shared memory and lets parent and child increment it.

Thread module: Creates two threads and protects the shared increment using a mutex.

5.5 Input and Output Design

Inputs are entered as textual commands. `create` and `open` additionally require a file name; `write` requires text; `shared` requires an integer operation count. Outputs report success/failure, file descriptor, process IDs, thread IDs, byte counts, shared values, and synchronization status.

CHAPTER -6 IMPLEMENTATION

6.1 File Management Implementation (CO-1)

The program stores the current descriptor in `fd`, initialized to -1. A value of -1 indicates that no file is currently open. `create` uses `open()` with `O_CREAT` and `O_TRUNC`, while `open` uses `O_RDWR`. `write()` stores user input, and `read()` first calls `lseek(fd, 0, SEEK_SET)` so that the subsequent read begins at the start of the file.

```
fd = open(filename, O_RDWR);
write(fd, buffer, strlen(buffer));
lseek(fd, 0, SEEK_SET);
bytes_read = read(fd, buffer, BUFFER_SIZE - 1);
close(fd);
```

6.2 Process and Pipe Implementation (CO-2 and CO-3)

The `process` command first creates a pipe and then calls `fork()`. The parent sends a formatted message containing the file name and descriptor. The child receives and prints the message. The parent calls `wait()` and reports the child's exit status.

```
pipe(pipefd);
pid_t pid = fork();
/* parent: write(pipefd[1], message, strlen(message)); */
/* child : read(pipefd[0], message, BUFFER_SIZE - 1); */
wait(&status);
```

6.3 Process Memory Implementation (CO-4)

The `memory` command starts with `value = 100`. After `fork()`, the child sets `value = 200`. The parent waits for the child and prints its own value, which remains 100 in the demonstrated execution. This is the observable behavior used to explain private process memory and copy-on-write.

6.4 Shared Data Implementation (CO-5)

The shared module allocates an integer with `mmap()`. `MAP_SHARED` makes the mapping visible to the parent and child after `fork()`. Each process performs `count` iterations of a non-atomic read-modify-write sequence.

```
shared_value = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE,
                    MAP_SHARED | MAP_ANONYMOUS, -1, 0);
/* each process: */
temp = *shared_value;
temp++;
*shared_value = temp;
```

Because the increment is not protected, the module demonstrates the conditions under which a

race condition can occur. The displayed execution produced Expected Value = 2000 and Final Value = 2000 for count = 1000, but a different interleaving can produce a different final value.

6.5 Thread Implementation (CO-6)

Two threads are created with the same `shared_value` address. The thread function locks a global mutex before incrementing and printing the value. The mutex is unlocked after the critical section. The main process joins both threads and then destroys the mutex.

```
pthread_mutex_lock(&mutex);
(*value)++;
printf("Shared Value: %d\n", *value);
pthread_mutex_unlock(&mutex);

pthread_join(thread1, NULL);
pthread_join(thread2, NULL);
```

6.6 Algorithms / Pseudocode

Algorithm 1: File I/O

1. Read file name.
2. Call open/create operation.
3. Store returned file descriptor.
4. For write, call `write()`.
5. For read, reset offset with `lseek()` and call `read()`.
6. For close, call `close()` and reset `fd` to -1.

Algorithm 2: Process + Pipe

1. Create pipe.
2. Call `fork()`.
3. Child closes write end and reads message.
4. Parent closes read end and writes file information.
5. Parent waits for child.

Algorithm 3: Memory

1. Initialize value to 100.
2. Call `fork()`.
3. Child changes its value to 200.
4. Parent waits and prints its value.

Algorithm 4: Shared Data

1. Create shared integer using `mmap()`.
2. Set shared value to zero.
3. Call `fork()`.
4. Parent and child each perform count read-modify-write increments.
5. Parent waits and displays expected and observed values.

Algorithm 5: Threads

1. Initialize mutex and shared value.
2. Create two threads.
3. Each thread locks mutex, increments shared value, prints it, and unlocks.
4. Join both threads.
5. Display final shared value and destroy mutex.

6.7 CO-wise Implementation Mapping

CO	Evidence in Code	Observed Demonstration
CO-1	open/read/write/close/lseek	FD 3, 34 bytes written/read, file content displayed
CO-2	fork/getpid/getppid/wait	Parent PID 713, child PID 720 in sample run
CO-3	pipe + parent write + child read	Child receives test.txt and FD 3
CO-4	fork + child variable update	Child value 200, parent value 100
CO-5	mmap MAP_SHARED + concurrent increments	Expected 2000, observed 2000 in sample run
CO-6	pthread_create + mutex + join	Two threads, final shared value 2

CHAPTER -7 EXECUTION AND DEMONSTRATION

7.1 Compilation in Linux

```
spoorthi@LAPTOP:~$ nano project1.c
spoorthi@LAPTOP:~$ gcc project1.c -o project1 -pthread
spoorthi@LAPTOP:~$ ./project1
```

7.2 Sample Execution

The following output is the demonstrated execution supplied for this project. Process and thread IDs are runtime-generated and may differ in another execution.

```
================================================
       LINUX FILE DESCRIPTOR AND FILE I/O SYSTEM
================================================

Enter command: create
Enter file name: test.txt

File created successfully!
File Name: test.txt

Enter command: open
Enter file name: test.txt

File opened successfully!
File Name      : test.txt
File Descriptor: 3

Enter command: write
Enter data to write: Hello from File Management System

Data written successfully!
Bytes written: 34

Enter command: read

Read successfully!
File Name      : test.txt
File Descriptor: 3
Bytes read     : 34

----- File Content -----
Hello from File Management System
------------------------

Enter command: process

========================================
             PARENT PROCESS
========================================
Parent Process ID: 713
Child Process ID : 720
Information sent successfully.
========================================

Waiting for child process to complete...

========================================
             CHILD PROCESS
========================================
Child Process ID : 720
Parent Process ID: 713

Message received:
```

```
File Name: test.txt
File Descriptor: 3
========================================

Child process completed successfully.
Child Process ID: 720
Child Exit Status: 0

Enter command: memory

========================================
             PROCESS MEMORY
========================================
Initial Value: 100

Child Process ID : 725
Child Value: 200

Parent Process ID: 713
Parent Value: 100

Process memory operation completed.
========================================

Enter command: shared
Enter number of operations: 1000

Child process completed.

========================================
              SHARED DATA
========================================
Expected Value: 2000
Final Value   : 2000
Data operations completed.
========================================

Enter command: execute

========================================
          THREAD MANAGEMENT
========================================
Main Process ID: 713

========================================
             THREAD EXECUTION
========================================
Thread ID: 127817501112000
Thread is executing...
Shared Value: 1
Thread completed successfully.
========================================

========================================
             THREAD EXECUTION
========================================
Thread ID: 127817492719296
Thread is executing...
Shared Value: 2
Thread completed successfully.
========================================

========================================
       THREAD SYNCHRONIZATION
========================================
Thread 1 joined successfully.
Thread 2 joined successfully.
Final Shared Value: 2
Execution completed successfully.
========================================

Enter command: close
```

```
File closed successfully!
File Name      : test.txt
Released FD    : 3

Enter command: exit
Program terminated successfully.
spoorthi@LAPTOP:~$
```

7.3 Demonstration of OS Concepts

Observed Output	Interpretation
File Descriptor: 3	Linux assigned FD 3 to the opened file in this execution.
Bytes written/read: 34	The File I/O calls transferred 34 bytes.
Parent/Child IDs	fork() created a child process and getpid/getppid reported IDs.
Message received	The child received file information through the pipe.
Child 200 / Parent 100	The variable was private to each process after fork().
Expected 2000 / Final 2000	The shared-data execution completed without a lost update in this run; race remains possible because no process-shared synchronization is used.
Final Shared Value: 2	The mutex protected the thread critical section and both threads were joined.

7.4 Output Analysis

The output demonstrates the complete flow from file management to concurrency. The file module establishes the file descriptor used by the process. The process module then transfers that information to a child through a pipe. The memory module contrasts private process state with the shared memory used in the shared module. Finally, the `execute` command demonstrates thread-level concurrency and synchronization using a mutex.

CHAPTER -8 RESULTS, CONCLUSION AND FUTURE SCOPE

8.1 Results

- File creation, opening, writing, reading, and closing were demonstrated successfully.
- A file descriptor was displayed and later released.
- A child process was created and parent/child process IDs were displayed.
- The parent successfully sent file information to the child through a pipe.
- The memory module showed child value 200 while the parent retained value 100 in the demonstrated run.
- Shared memory was created and accessed by both processes; the demonstrated run produced the expected value 2000.
- Two POSIX threads were created, synchronized using a mutex, joined successfully, and produced final shared value 2.

8.2 Learning Outcomes

- Understanding the role of file descriptors in Linux File I/O.
- Understanding `fork()`, parent/child execution, process IDs, and `wait()`.
- Understanding pipe-based IPC.
- Understanding private process memory and copy-on-write behavior.
- Understanding shared memory and the read-modify-write race condition.
- Understanding thread concurrency, critical sections, mutexes, and thread joining.

8.3 Conclusion

The Linux File Descriptor and File I/O Management System successfully combines six major Operating Systems and Systems Programming concepts in one C application. The project provides direct, observable demonstrations of file descriptors and File I/O, process creation and IDs, pipe communication, process memory behavior, shared data, race conditions, POSIX threads, and synchronization. The command-driven design makes each concept independently executable while keeping the overall project integrated.

8.4 Future Scope

- Support simultaneous management of multiple opened files and maintain a descriptor table.

- Add robust input validation for file names and command arguments.
- Add explicit process-shared synchronization to provide a separate safe shared-counter mode.
- Add a controlled race-condition demonstration that compares unsynchronized and synchronized shared increments.
- Add signal handling and more detailed error reporting.
- Add a graphical or web-based visualization layer while retaining the Linux system-call backend.

CHAPTER -9 REFERENCES

- Operating Systems and Systems Programming (25CS2104E), 2026–27 Term-I, Project Problem Statement Submission Form.
- GNU/Linux C library and POSIX API documentation for `open()`, `read()`, `write()`, `close()`, `lseek()`, `fork()`, `pipe()`, `wait()`, `mmap()`, and pthread APIs.
- Linux manual pages (man pages) for the system calls and POSIX functions used in the implementation.
- The project source code: `project1.c`, supplied for this report.

APPENDIX A SOURCE CODE

The following is the project source code supplied for this report. It is included as the implementation appendix and corresponds to the execution described in Chapter 7.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <pthread.h>

#define BUFFER_SIZE 1024

pthread_mutex_t mutex;

void *thread_function(void *arg)
{
    int *value = (int *)arg;

    pthread_mutex_lock(&mutex);

    printf("\n=======================================\n");
    printf("             THREAD EXECUTION\n");
    printf("=======================================\n");
    printf("Thread ID: %lu\n", (unsigned long)pthread_self());
    printf("Thread is executing...\n");

    (*value)++;

    printf("Shared Value: %d\n", *value);
    printf("Thread completed successfully.\n");
    printf("=======================================\n");

    pthread_mutex_unlock(&mutex);

    return NULL;
}

int main()
{
    int fd = -1;
    char filename[256];
    char command[50];
    char buffer[BUFFER_SIZE];

    printf("=================================================\n");
    printf("       LINUX FILE DESCRIPTOR AND FILE I/O SYSTEM\n");
    printf("=================================================\n");

    while (1)
    {
        printf("\nEnter command: ");
        scanf("%49s", command);

        if (strcmp(command, "create") == 0)
        {
            int create_fd;

            printf("Enter file name: ");
            scanf("%255s", filename);

            create_fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

            if (create_fd == -1)
            {
                perror("File creation failed");
                continue;
            }

            printf("\nFile created successfully!\n");
```

```
            printf("File Name: %s\n", filename);

            close(create_fd);
        }

        else if (strcmp(command, "open") == 0)
        {
            if (fd != -1)
            {
                printf("A file is already open.\n");
                printf("Current File Descriptor: %d\n", fd);
                continue;
            }

            printf("Enter file name: ");
            scanf("%255s", filename);

            fd = open(filename, O_RDWR);

            if (fd == -1)
            {
                perror("Open failed");
                continue;
            }

            printf("\nFile opened successfully!\n");
            printf("File Name      : %s\n", filename);
            printf("File Descriptor: %d\n", fd);
        }

        else if (strcmp(command, "write") == 0)
        {
            if (fd == -1)
            {
                printf("No file is open. Please use 'open' first.\n");
                continue;
            }

            printf("Enter data to write: ");
            getchar();

            if (fgets(buffer, BUFFER_SIZE, stdin) == NULL)
            {
                printf("Input error.\n");
                continue;
            }

            ssize_t bytes_written = write(fd, buffer, strlen(buffer));

            if (bytes_written == -1)
            {
                perror("Write failed");
                continue;
            }

            printf("\nData written successfully!\n");
            printf("Bytes written: %ld\n", bytes_written);
        }

        else if (strcmp(command, "read") == 0)
        {
            if (fd == -1)
            {
                printf("No file is open. Please use 'open' first.\n");
                continue;
            }

            if (lseek(fd, 0, SEEK_SET) == -1)
            {
                perror("File position error");
                continue;
            }

            ssize_t bytes_read = read(fd, buffer, BUFFER_SIZE - 1);

            if (bytes_read == -1)
            {
```

```
            perror("Read failed");
            continue;
        }

        buffer[bytes_read] = '\0';

        printf("\nRead successfully!\n");
        printf("File Name      : %s\n", filename);
        printf("File Descriptor: %d\n", fd);
        printf("Bytes read     : %ld\n", bytes_read);

        printf("\n----- File Content -----\n");
        printf("%s", buffer);
        printf("------------------------\n");
    }

    else if (strcmp(command, "process") == 0)
    {
        if (fd == -1)
        {
            printf("No file is open. Please use 'open' first.\n");
            continue;
        }

        int pipefd[2];

        if (pipe(pipefd) == -1)
        {
            perror("Pipe creation failed");
            continue;
        }

        pid_t pid = fork();

        if (pid == -1)
        {
            perror("Process creation failed");
            close(pipefd[0]);
            close(pipefd[1]);
            continue;
        }

        if (pid == 0)
        {
            close(pipefd[1]);

            char message[BUFFER_SIZE];

            ssize_t bytes_read =
                read(pipefd[0], message, BUFFER_SIZE - 1);

            if (bytes_read == -1)
            {
                perror("Pipe read failed");
                close(pipefd[0]);
                exit(1);
            }

            message[bytes_read] = '\0';

            printf("\n=======================================\n");
            printf("             CHILD PROCESS\n");
            printf("=======================================\n");
            printf("Child Process ID : %d\n", getpid());
            printf("Parent Process ID: %d\n", getppid());
            printf("\nMessage received:\n");
            printf("%s", message);
            printf("=======================================\n");

            close(pipefd[0]);
            exit(0);
        }
        else
        {
            close(pipefd[0]);

            char message[BUFFER_SIZE];
```

```
            snprintf(
                message,
                BUFFER_SIZE,
                "File Name: %s\nFile Descriptor: %d\n",
                filename,
                fd
            );

            write(pipefd[1], message, strlen(message));

            printf("\n=======================================\n");
            printf("            PARENT PROCESS\n");
            printf("=======================================\n");
            printf("Parent Process ID: %d\n", getpid());
            printf("Child Process ID : %d\n", pid);
            printf("Information sent successfully.\n");
            printf("=======================================\n");

            close(pipefd[1]);

            printf("\nWaiting for child process to complete...\n");

            int status;
            pid_t finished_pid = wait(&status);

            if (finished_pid == pid && WIFEXITED(status))
            {
                printf("\nChild process completed successfully.\n");
                printf("Child Process ID: %d\n", finished_pid);
                printf("Child Exit Status: %d\n",
                       WEXITSTATUS(status));
            }
        }
    }

    else if (strcmp(command, "memory") == 0)
    {
        int value = 100;

        printf("\n=======================================\n");
        printf("             PROCESS MEMORY\n");
        printf("=======================================\n");
        printf("Initial Value: %d\n", value);

        pid_t pid = fork();

        if (pid == -1)
        {
            perror("Process creation failed");
            continue;
        }

        if (pid == 0)
        {
            value = 200;

            printf("\nChild Process ID : %d\n", getpid());
            printf("Child Value: %d\n", value);

            exit(0);
        }
        else
        {
            wait(NULL);

            printf("\nParent Process ID: %d\n", getpid());
            printf("Parent Value: %d\n", value);
            printf("\nProcess memory operation completed.\n");
            printf("=======================================\n");
        }
    }

    else if (strcmp(command, "shared") == 0)
    {
        int *shared_value;
```

```
            shared_value = mmap(
                NULL,
                sizeof(int),
                PROT_READ | PROT_WRITE,
                MAP_SHARED | MAP_ANONYMOUS,
                -1,
                0
            );

            if (shared_value == MAP_FAILED)
            {
                perror("Shared memory creation failed");
                continue;
            }

            *shared_value = 0;

            int count;

            printf("Enter number of operations: ");
            scanf("%d", &count);

            pid_t pid = fork();

            if (pid == -1)
            {
                perror("Process creation failed");
                munmap(shared_value, sizeof(int));
                continue;
            }

            if (pid == 0)
            {
                for (int i = 0; i < count; i++)
                {
                    int temp = *shared_value;
                    temp++;
                    *shared_value = temp;
                }

                printf("\nChild process completed.\n");
                exit(0);
            }
            else
            {
                for (int i = 0; i < count; i++)
                {
                    int temp = *shared_value;
                    temp++;
                    *shared_value = temp;
                }

                wait(NULL);

                printf("\n========================================\n");
                printf("              SHARED DATA\n");
                printf("========================================\n");
                printf("Expected Value: %d\n", count * 2);
                printf("Final Value   : %d\n", *shared_value);
                printf("Data operations completed.\n");
                printf("========================================\n");

                munmap(shared_value, sizeof(int));
            }
        }

        else if (strcmp(command, "execute") == 0)
        {
            pthread_t thread1;
            pthread_t thread2;

            int shared_value = 0;

            if (pthread_mutex_init(&mutex, NULL) != 0)
            {
                printf("Mutex initialization failed.\n");
                continue;
```

```
            }

            printf("\n=======================================\n");
            printf("          THREAD MANAGEMENT\n");
            printf("=======================================\n");
            printf("Main Process ID: %d\n", getpid());

            if (pthread_create(
                    &thread1,
                    NULL,
                    thread_function,
                    &shared_value) != 0)
            {
                printf("Thread creation failed.\n");
                pthread_mutex_destroy(&mutex);
                continue;
            }

            if (pthread_create(
                    &thread2,
                    NULL,
                    thread_function,
                    &shared_value) != 0)
            {
                printf("Thread creation failed.\n");
                pthread_join(thread1, NULL);
                pthread_mutex_destroy(&mutex);
                continue;
            }

            pthread_join(thread1, NULL);
            pthread_join(thread2, NULL);

            printf("\n=======================================\n");
            printf("       THREAD SYNCHRONIZATION\n");
            printf("=======================================\n");
            printf("Thread 1 joined successfully.\n");
            printf("Thread 2 joined successfully.\n");
            printf("Final Shared Value: %d\n", shared_value);
            printf("Execution completed successfully.\n");
            printf("=======================================\n");

            pthread_mutex_destroy(&mutex);
        }

        else if (strcmp(command, "close") == 0)
        {
            if (fd == -1)
            {
                printf("No file is currently open.\n");
                continue;
            }

            if (close(fd) == -1)
            {
                perror("Close failed");
                continue;
            }

            printf("\nFile closed successfully!\n");
            printf("File Name      : %s\n", filename);
            printf("Released FD    : %d\n", fd);

            fd = -1;
        }

        else if (strcmp(command, "exit") == 0)
        {
            if (fd != -1)
            {
                close(fd);
                printf("\nOpen file closed automatically.\n");
            }

            printf("Program terminated successfully.\n");
            break;
        }
```

```
        else
        {
            printf("\nInvalid command.\n");
            printf("Use: create, open, write, read, process, memory, shared, execute, close,
exit\n");
        }
    }

    return 0;
}
```